

Sisteme și algoritmi distribuiți

Curs 6

Defecte

Defecte

- Un număr sporit de noduri implică o probabilitate de defecte în creștere.
- Gravitatea defectului depinde de aplicație: sistem de control al traficului aerian vs sistem de gaming.
- Sursele defectelor la nivel de nod:
 - Erori: design, fabricație, programare
 - Accidente fizice
 - Condiții de mediu dure
 - Date de intrare neasteptate
 - Etc.

Modelarea defectelor

Pentru a evita restartarea algoritmilor de fiecare dată când apare un defect (sau adăugarea permanentă de resurse), sunt necesare *schemele (algoritmii) tolerante la defecte*.

Principalele modele:

- Defect Crash
- Defect Bizantin

Nodurile care nu suferă defect se numesc noduri corecte.

Redundanță

- Redundanța informației:

- Adaugă extra biți pentru recuperarea datelor; tehnici ECC, coduri Hamming; checksum / CRC – pentru detecția alterării mesajelor.

- Redundanța de timp:

- O acțiune executată va fi repetată dacă este necesar (E.g. Re-transmiterea pachetelor, tranzacții atomice, etc)

- Redundanța fizică:

- Echipament extra adăugat în sistem pentru a tolera eventualele defecte.
- Două moduri de organizare:
 - a) Active replication
 - b) Primary backup

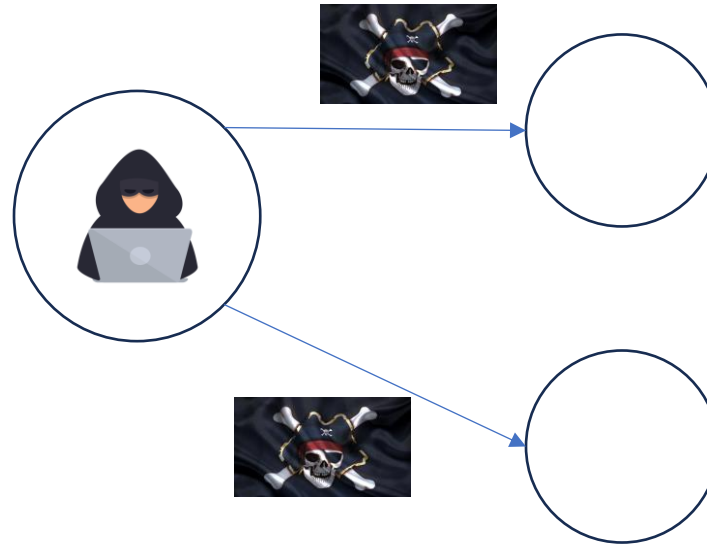
Defectul Crash

- La momentul defectului, nodul:
 - Oprește execuția locală a iterațiilor
 - Nu primește mesaje
 - Nu trimite mesaje
- În perspectiva cea mai simplistă: nodul nu reia activitatea niciodată.
- Sub-clase de Crash:
 - Crash-stop
 - Omisiune de mesaje
 - Crash cu revenire

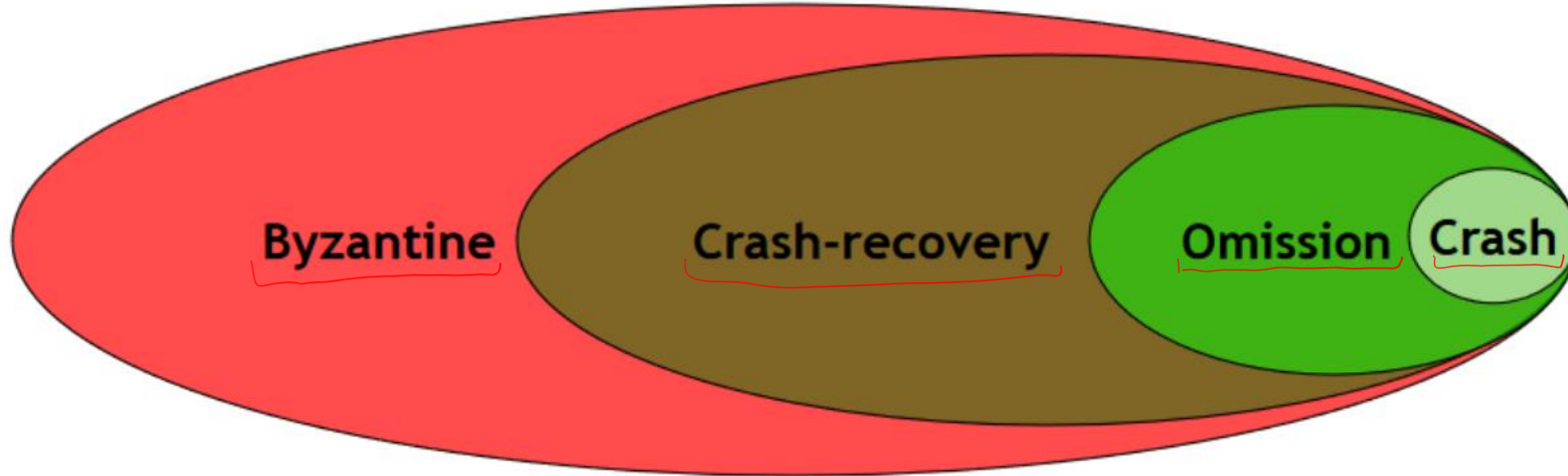
Defect Bizantin

Sub defect bizantin nodurile se comportă malițios, perturbând activitatea întregului sistem (e.g. comportament arbitrar):

- Livrează mesaje atipice execuției algoritmului local
- Actualizează starea după reguli atipice execuției algoritmului local



Ierarhie



Consens distribuit (cu procese defecte)

Starea (valoarea) uniformă a nodurilor unui sistem distribuit.

Condiția de acord: Nu există două procese care decid valori diferite.

Condiția de validitate: Dacă valoarea inițială a proceselor este v , atunci consensul se atinge cu valoarea uniformă v .

Condiția de terminare (algorithm): Într-un algoritm de consens, orice nod corect din sistem va decide eventual la un moment de timp.

În general, decizia se reduce la evaluarea funcției de consens $f(\cdot)$ în $x(0)$.

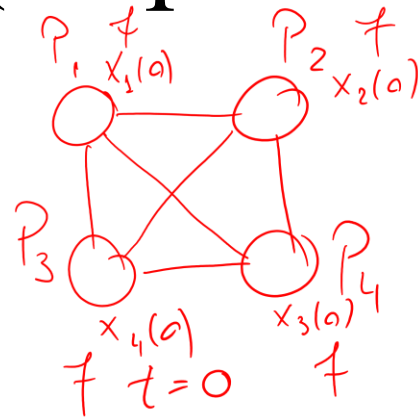
$x_i(T) \neq f(x(0))$, i nod corect

Consens distribuit (cu procese defecte)

Ipoteze:

- Graf complet (nedirectat)
- Un nod poate fi corect sau defect *Crash* (încetează funcționarea)
- Dacă un nod intră în starea de defect, rămâne în ea până la finalul algoritmului

Urmărim atingerea consensului distribuit sub maxim $s \geq 0$ defecte *Crash*.



Algoritm **FloodSet**($f()$): *consens distribuit*

M_i : - int id (id propriu)
 - int v (token, inițial egal cu x_i)
 funcție obiectiv $f()$
 - int t , integer, inițial 0

Funcție transformare nod $i()$:

1. **Bcast**($\langle x_i(0), id_i \rangle$)
2. Fie U mulțimea mesajelor $\langle v_j, id_j \rangle$ primite restul nodurilor
3. $M_i(t+1) = M_i(t) \cup U$ *$O(m)$*
4. Fie $V_i(t+1)$ mulțimea valorilor v_j din $M_i(t+1)$
5. Fie $I_i(t+1)$ mulțimea id-urilor id_j din $M_i(t+1)$ *minim $M_i(t+1)$ corectă*
6. **Return** $f(V_i(t))$

Time: $O(1)$ publica (fără defect)

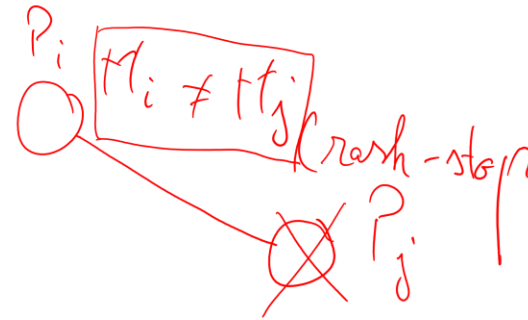
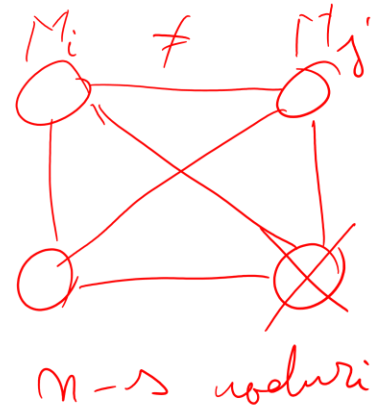
Algoritm FloodSet pentru clică

Algoritm **FloodSet**($f()$):

M_i : - int id (id propriu)
- int v (token, inițial egal cu x_i)
- funcție obiectiv $f()$
- int t , integer, inițial 0

Funcție transformare nod i ():

1. **Bcast**($\langle x_i(0), id_i \rangle$)
2. Fie U mulțimea mesajelor $\langle v_j, id_j \rangle$ primite de la restul nodurilor $M_i = \{t_j\}$
3. $M_i(t+1) = M_i(t) \cup U$
4. Fie $V_i(t+1)$ mulțimea valorilor v_j din $M_i(t+1)$
5. Fie $I_i(t+1)$ mulțimea id-urilor id_j din $M_i(t+1)$
6. **Return** $f(V_i(t))$



← $j \in \mathcal{N}_i^-$ cade la momentul t , atunci $M_j(t)$ nu va ajunge la P_i la momentul $t+1$, $M_j(t) \neq M_i(t)$

← $V_i \neq V_k$

Algoritm FloodSet s –robust (clică, defect Crash)

nr. crash admin

Algoritm FloodSet($f, x(0), s$):

M_i : - int id (id propriu)

- int v (token, inițial egal cu x_i)

- funcție obiectiv $f()$

- int t , integer, inițial 0

Funcție transformare nod i ():

1. Bcast($M_i(t)$)

2. Fie U mulțimea mesajelor $\langle v_j, id_j \rangle$ primite de la restul nodurilor

3. $M_i(t+1) = M_i(t) \cup U$

4. Fie $V_i(t+1)$ mulțimea valorilor v_j din $M_i(t+1)$

5. Fie $I_i(t+1)$ mulțimea id-urilor id_j din $M_i(t+1)$

6. If ($t > s + 1$):

1. Return $f(M_i(t))$

7. $t := t + 1$

- Paradigma de ‘robustificare’ a algoritmilor distribuiți
- Se realizează $s + 1$ iterații (s explicit)
- **Lemma 1.** Dacă există o iterație t în care nu există defect, atunci $M_i(t) = M_j(t)$ pentru orice noduri i și j corecte la momentul t .
- **Lemma 2.** Dacă $M_i(t) = M_j(t)$ pentru orice noduri i și j corecte. Atunci pentru orice $t \leq t' \leq s + 1$ avem $M_i(t') = M_j(t')$, oricare ar fi nodurile corecte i și j la momentul t' .
- **Teorema.** FloodSet s –robust rezolvă problema de consens pentru defecte de tip *Crash*.
- Principalul argument: avem s defecte, de aceea după $s + 1$ iterații va exista cel puțin o iterație t în care nu există defect. Lemma 1 implică $M_i(t) = M_j(t)$ pentru orice noduri i și j corecte la momentul t . Lemma 2 implică $M_i(s + 1) = M_j(s + 1)$ pentru orice noduri i și j corecte la momentul $s + 1$.

Algoritm FloodSet s –robust (clică, defect Crash)

Algoritm FloodSet($f, x(0), s$):

M_i : - int id (id propriu)

- int v (token, inițial egal cu x_i)

- funcție obiectiv $f()$

- int t , integer, inițial 0

Funcție transformare nod i ():

1. **Bcast**($M_i(t)$)

2. Fie U mulțimea mesajelor $\langle v_j, id_j \rangle$ primite restul nodurilor

3. $M_i(t+1) = M_i(t) \cup U$

4. Fie $V_i(t+1)$ mulțimea valorilor v_j din $M_i(t+1)$

5. Fie $I_i(t+1)$ mulțimea id-urilor id_j din $M_i(t+1)$

6. **If** ($t > s + 1$):

 1. **Return** $f(M_i(t))$

7. $t := t + 1$

- Operația de Broadcast costă n mesaje.

- Complexitate timp: $s + 1$ *optimală*

- Complexitate mesaj: $(s + 1)n$

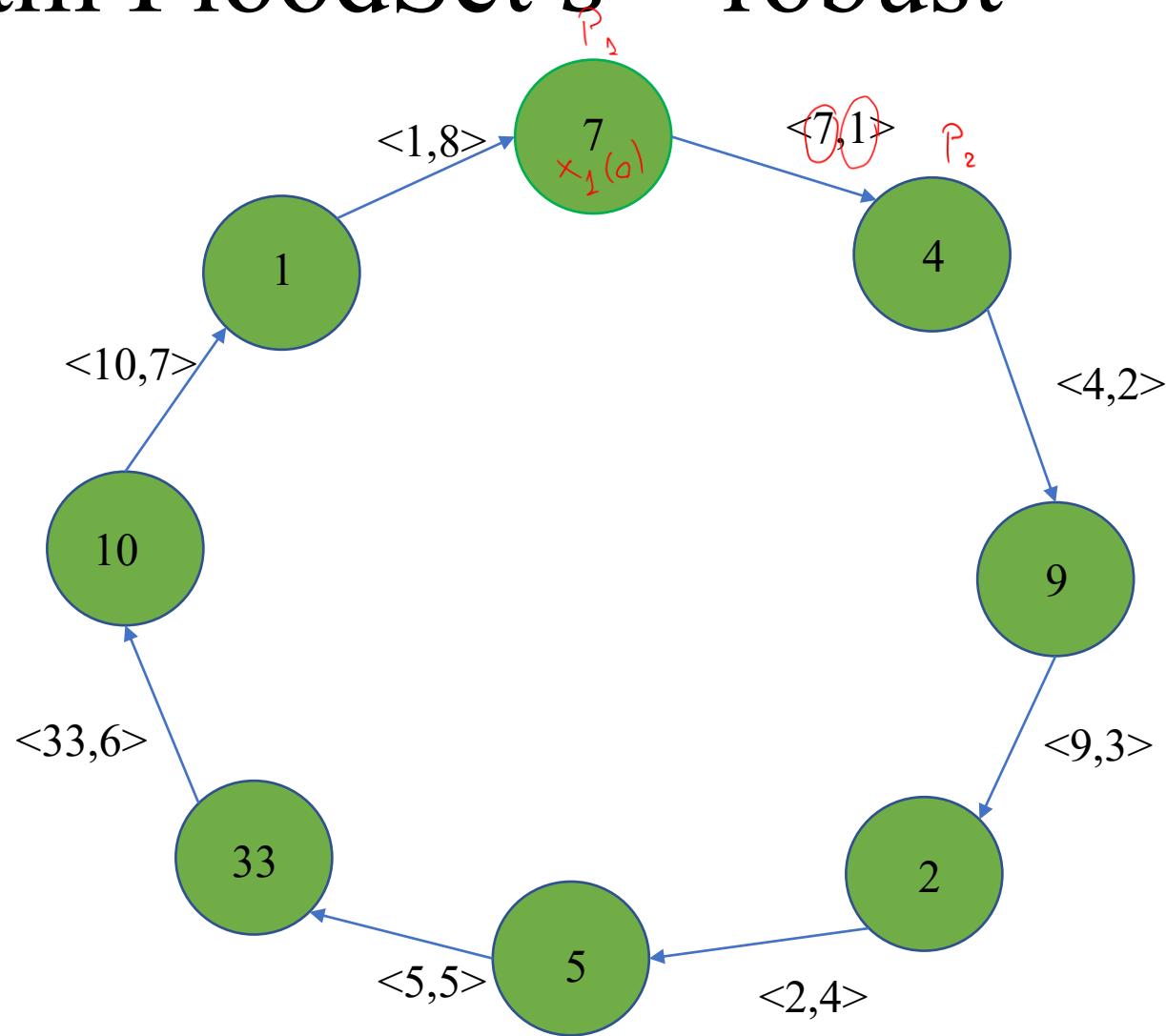
- Rezultat margine inferioară (informal)[Lynch]: orice algoritm s -robust la defecte Crash execută minim $s + 1$ iterații.

- Cum generalizăm pentru grafuri conexe generale (nu neapărat complete)?

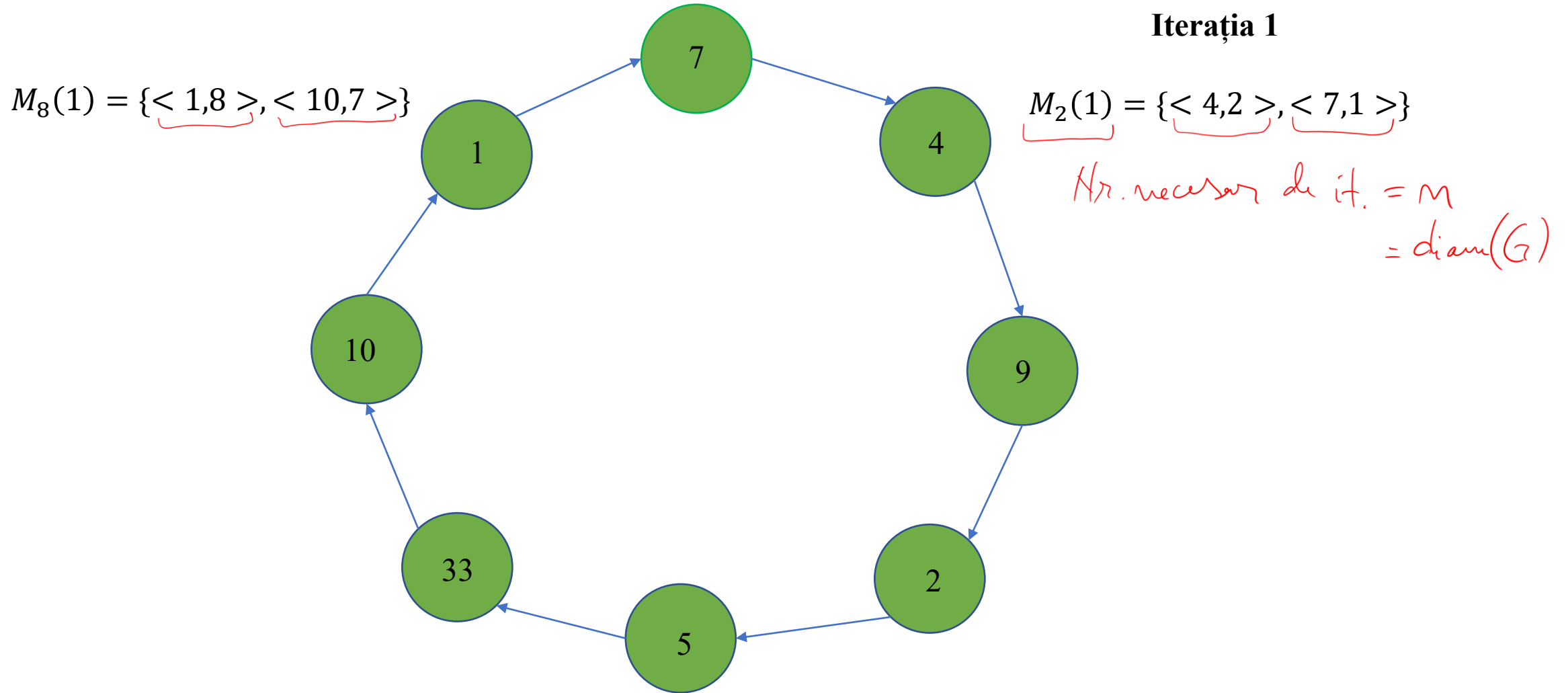
Bcast $\rightarrow$ către vecini

Algorithm FloodSet s –robust

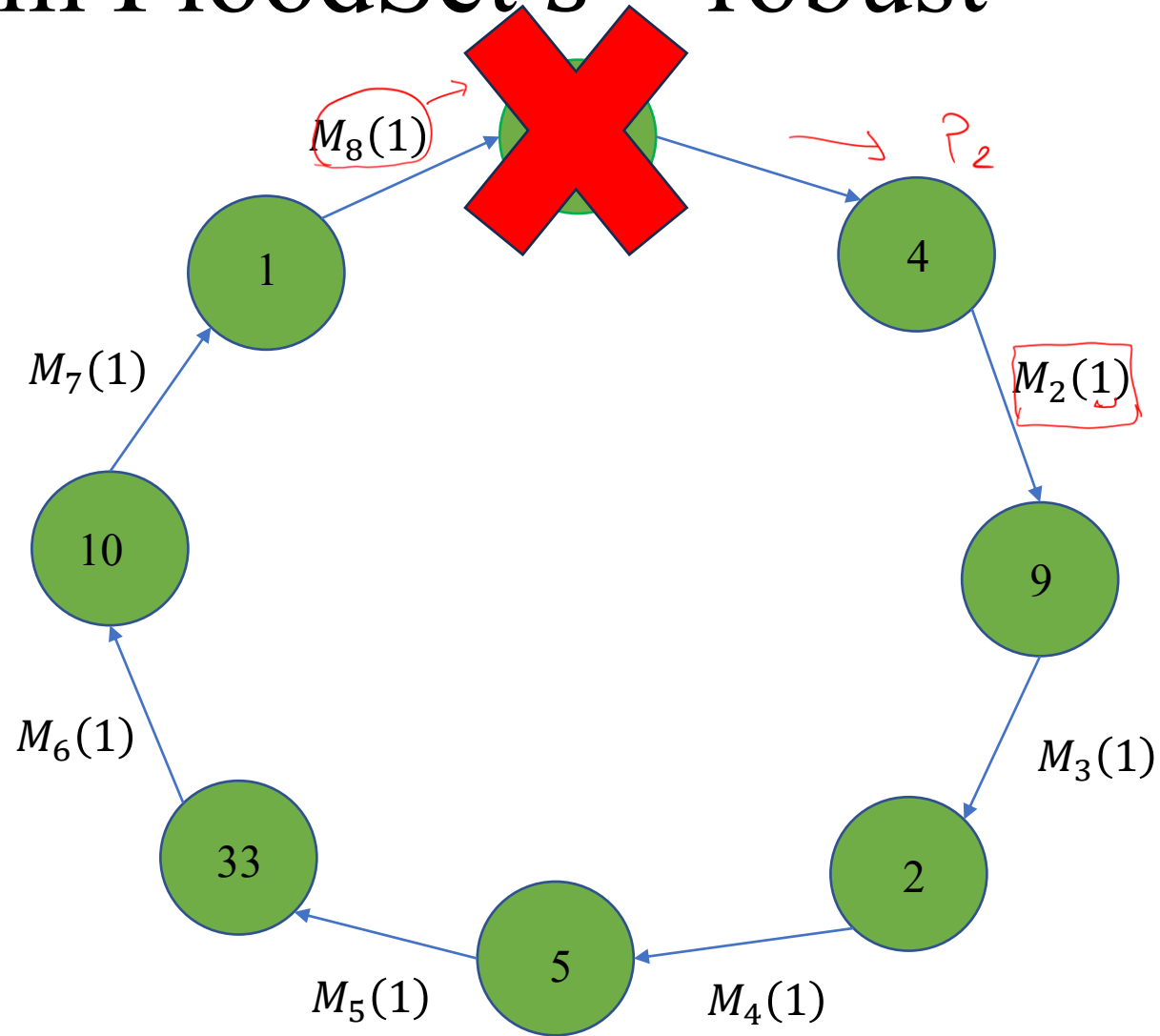
Iterația 1



Algoritm FloodSet s –robust



Algorithm FloodSet s –robust



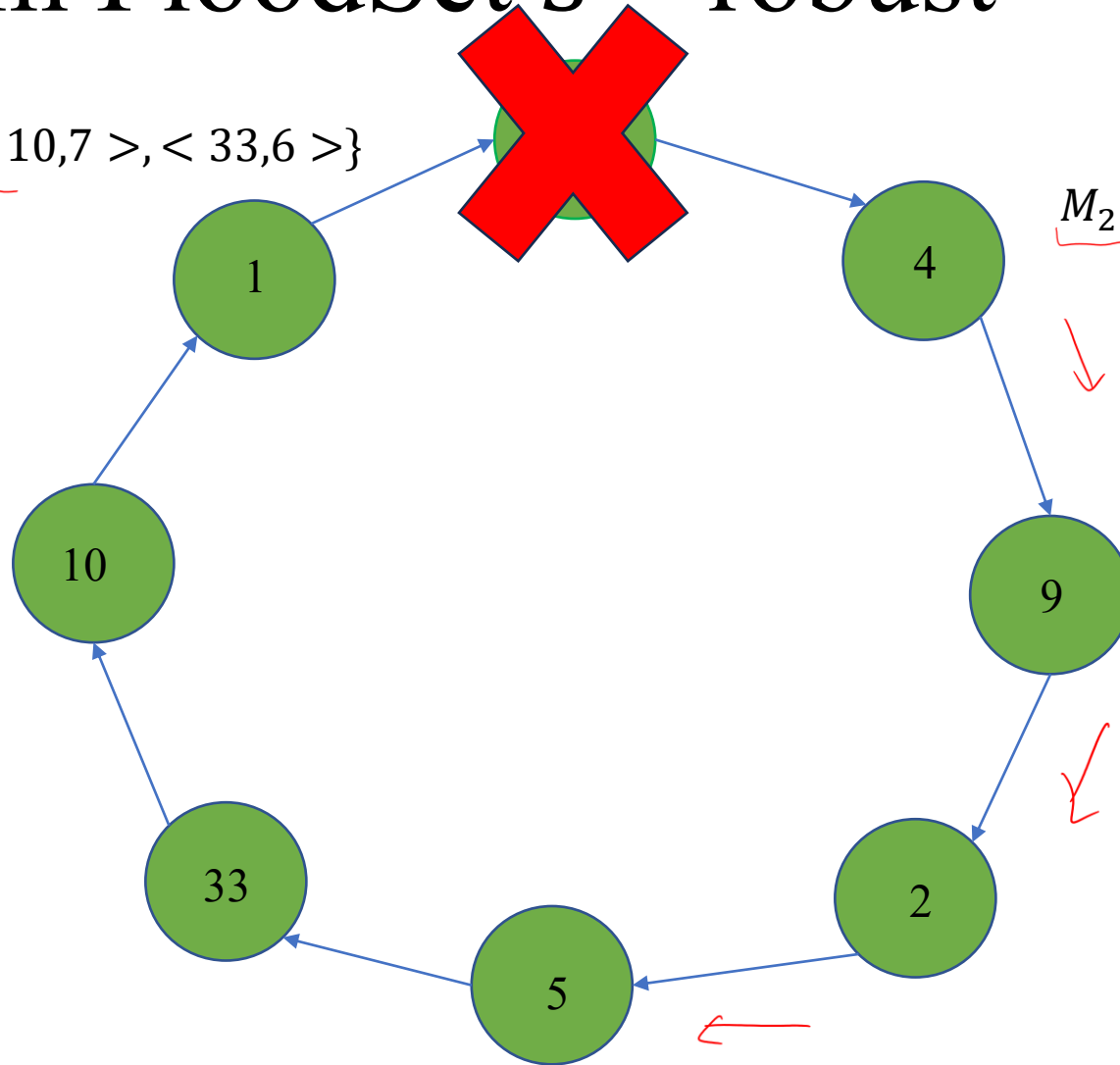
Iterația 2

Algoritm FloodSet s –robust

$$M_8(1) = \{< 1,8 >, < 10,7 >, < 33,6 >\}$$

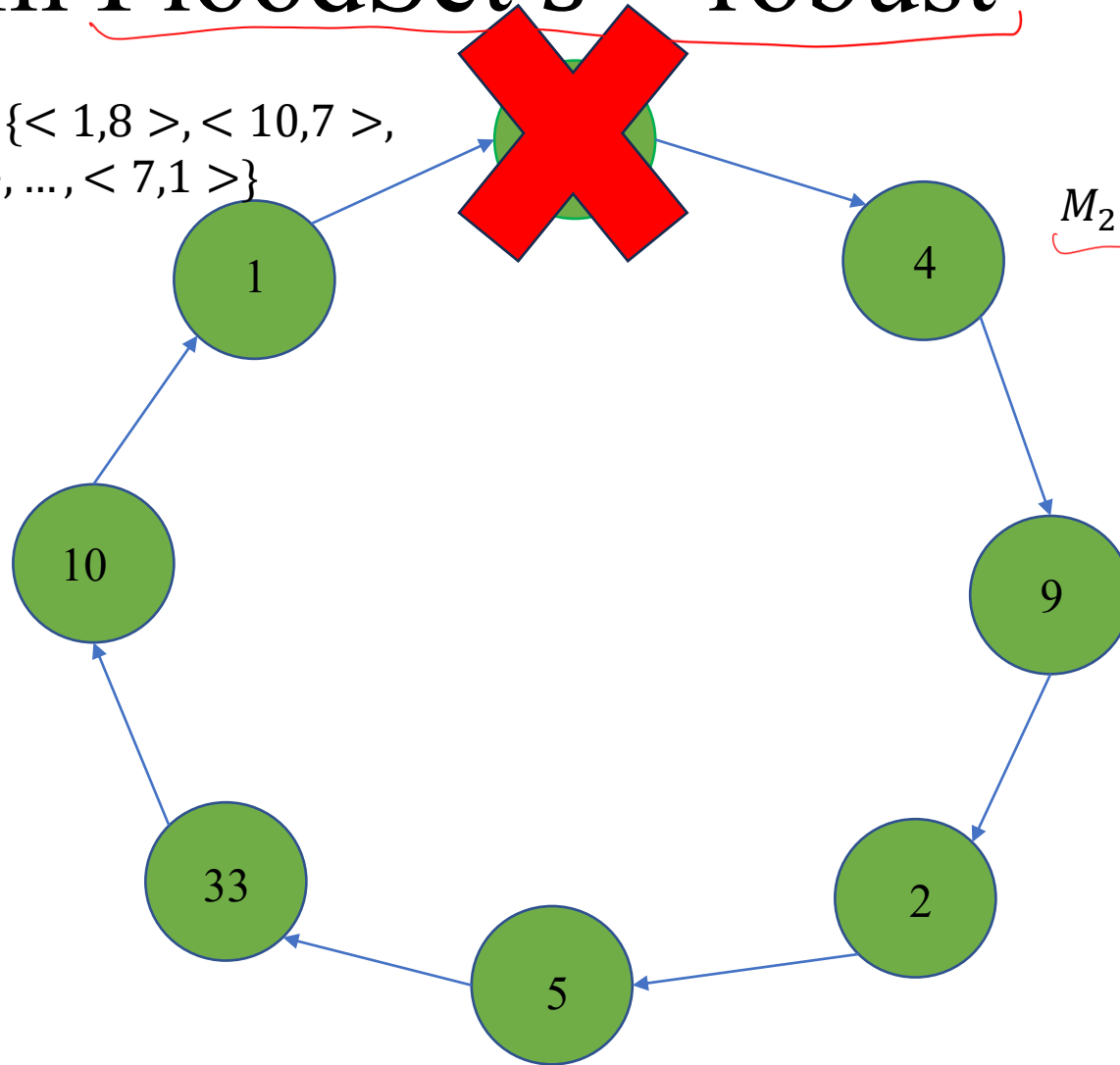
Iterația 2

$$M_2(1) = \{< 4,2 >, < 7,1 >\}$$



Algorithm FloodSet s –robust

$$\underline{M_8(7)} = \{ \langle 1, 8 \rangle, \langle 10, 7 \rangle, \langle 33, 6 \rangle, \dots, \langle 7, 1 \rangle \}$$



Iterația 7

$$\underline{M_2(7)} = \{ \langle 4, 2 \rangle, \langle 7, 1 \rangle \}$$

Conectivitate

$$\text{conn}(G) = 1$$

$$\Delta = 1$$

Consens distribuit (cu procese defecte)

Ipoteze:

- Graf complet conex (nedirectat). Conectivitatea grafului G , $conn(G)$ = numărul minim de noduri care, o dată eliminate din graf, va rezulta un graf neconectat.
- $s < conn(G)$
- Un nod poate fi corect sau defect *Crash* (încetează funcționarea)
- Dacă un nod intră în starea de defect, rămâne în ea până la finalul algoritmului

Urmărim atingerea consensului distribuit sub maxim $s \geq 0$ defecte *Crash*.

Algoritm FloodSet s –robust (conex, defect Crash)

Algoritm FloodSet($f()$): $\Delta < \text{conn}(G)$

M_i : - int id (id propriu)

- int v (token, inițial egal cu x_i)

- funcție obiectiv $f()$

- int t , integer, inițial 0

Funcție transformare nod i ():

1. Fie U mulțimea mesajelor $\langle v_j, id_j \rangle$ primite de la $\mathcal{N}_i^-$
2. $M_i(t+1) = M_i(t) \cup U$
3. Fie $V_i(t+1)$ mulțimea valorilor v_j din $M_i(t+1)$
4. Fie $I_i(t+1)$ mulțimea id-urilor id_j din $M_i(t+1)$
5. **If** $(t > (s+1)diam)$:
 1. **Return** $f(M_i(t))$
6. **Else**: send($M_i(t+1)$, $\mathcal{N}_i^+$)
7. $t := t+1$

- Ipoteza $s < \text{conn}(G)$ garantează că graful rezultat în urma defectelor rămâne conex.
- Se realizează $s+1$ seturi de $diam(G)$ iterații.
- Convergența folosește aceleași argumente ca în cazul clicii; în principal, după $s+1$ seturi de $diam(G)$ iterații, există cel puțin unul în care niciun nod nu are defect. Însă $diam(G)$ iterații sunt suficiente pentru a atinge consensul între nodurile corecte.

Algoritm Flooding pentru consens

Algoritm Flooding de medie prezentat anterior se exprimă recurent prin actualizarea liniară:

$$\underbrace{x_i(t+1)} = \underbrace{a_{ii}x_i(t) + \sum_{j \in \mathcal{N}_i^-} a_{ij}x_j(t)} \quad \forall i \quad \sum_{j \in \mathcal{N}_i^-} a_{ij} + a_{ii} = 1$$

Deci $x_i(t+1) = a_i^T x(t)$, iar dinamica stărilor sistemului este:

$$x(t+1) = Ax(t),$$

unde

$$A = \begin{bmatrix} a_1^T \\ \dots \\ a_n^T \end{bmatrix} \in R^{n \times n}, x(t) = \begin{bmatrix} x_1(t) \\ \dots \\ x_n(t) \end{bmatrix} \in R^n.$$

Problemă

Fie graful $G = (V, E)$ cu matricea de adiacență ponderată $A \in R^{5 \times 5}$.

- Care este valoarea de consens asimptotic a algoritmului Flooding de medie?
- Presupunem că la momentul de timp $t = 3$, nodurile 2 și 3 suferă defect de tip *Crash*. Cum se schimbă valoarea de consens asimptotic a algoritmului Flooding de medie în acest caz?

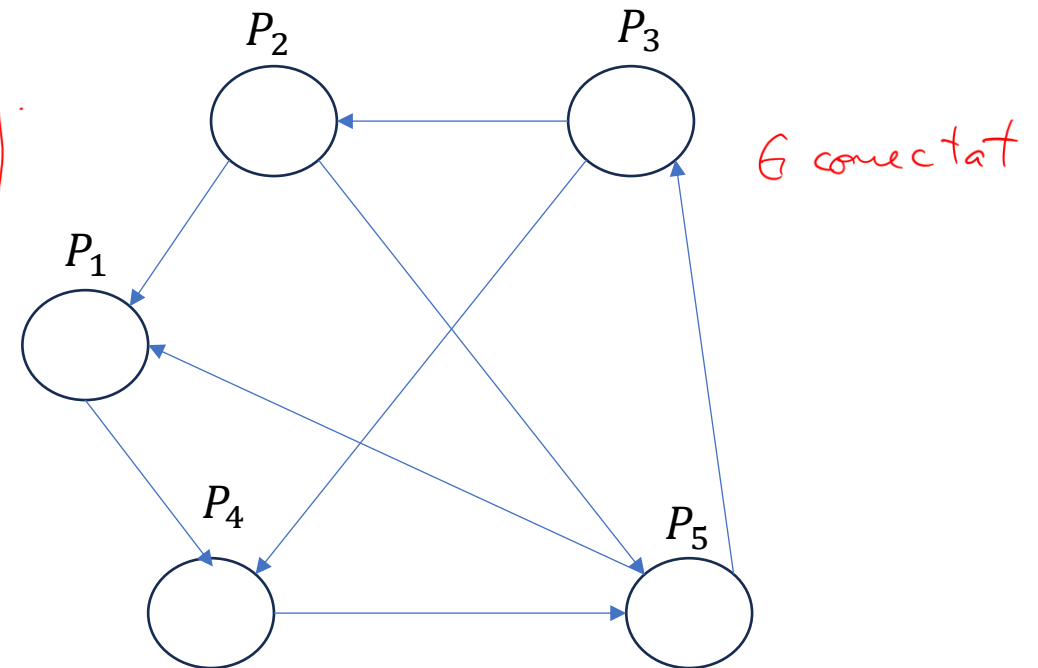
A - mat. ad. cu pondera uniformă

$$x(0), x(1) = Ax(0) \Rightarrow x(2) = Ax(1) \Rightarrow \dots \Rightarrow x(\infty) = \begin{bmatrix} c \\ \vdots \\ c \end{bmatrix} = c \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}$$

$$c = w^T \cdot x(0) = \frac{1}{n} \sum_{i=1}^n x_i(0)$$

w - vect. pr. la stânga al. mat. A asociat v.p. 1.

$$A = \begin{bmatrix} 1/3 & 1/3 & 0 & 0 & 1/3 \\ 0 & 1/2 & 1/2 & 0 & 0 \\ 0 & 0 & 1/2 & 0 & 1/2 \\ 1/3 & 0 & 1/3 & 1/3 & 0 \\ 0 & 1/3 & 0 & 1/3 & 1/3 \end{bmatrix}; \quad w^T A = w$$



Problemă

$$W^T A = W^T \Leftrightarrow \begin{cases} \frac{w_1}{3} + \frac{w_4}{3} = w_1 \\ \frac{w_1}{3} + \frac{w_2}{2} + \frac{w_5}{3} = w_2 \\ \frac{w_2}{2} + \frac{w_3}{2} + \frac{w_4}{3} = w_3 \\ \frac{w_4}{3} + \frac{w_5}{3} = w_4 \\ \frac{w_1}{3} + \frac{w_3}{2} + \frac{w_5}{3} = w_5 \end{cases} \quad \underbrace{W^T}_{[w_1 \ w_2 \ w_3 \ w_4 \ w_5]} \begin{bmatrix} 1/3 \\ 0 \\ 0 \\ 1/3 \\ 0 \end{bmatrix} = [w_1 \ w_2 \ \dots \ w_5]$$

$$w_4 = 2w_1 \Leftrightarrow w_2 = \frac{w_4}{2} \Rightarrow w_1 = \frac{w_5}{4}$$

$$w_4 = \frac{w_5}{2}$$

$$\frac{w_5}{12} + \frac{w_5}{3} = \frac{w_2}{2} \Rightarrow w_2 = \frac{5}{6}w_5$$

$$\frac{5w_5}{12} + \frac{w_5}{2} = \frac{w_3}{2} \Rightarrow w_3 = \left(\frac{5}{6} + \frac{1}{3}\right)w_5 = \frac{7}{6}w_5$$

$$x(0) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \rightarrow C = (w^*)^T \cdot x(0) = \frac{1}{4\rho} = \left\{ \frac{1}{4 \sqrt{\left(\frac{1}{4}\right)^2 + \dots + 1^2}} \right\}$$

e) $t=3 \Rightarrow$  $\Rightarrow A = \begin{bmatrix} 1/2 & 0 & 1/2 \\ 1/2 & 1/2 & 0 \\ 0 & 1/2 & 1/2 \end{bmatrix}$, dublu stoh.

$$W^T A = W^T, \quad W^* = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

$$x(0) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad x(1) = A x(0) = \begin{bmatrix} 1/3 \\ 0 \\ 0 \\ 1/3 \end{bmatrix}, \quad x(2) = \begin{bmatrix} 1/9 \\ 0 \\ 0 \\ 1/9 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1/9 \end{bmatrix} = \begin{bmatrix} 1/9 \\ 0 \\ 0 \\ 2/9 \end{bmatrix} \Rightarrow x(3) = \begin{bmatrix} 1/9 \\ 2/9 \\ 1/9 \end{bmatrix}, \quad C = \frac{4}{9} \cdot \frac{1}{\sqrt{3}} = \frac{4}{9\sqrt{3}}$$

$$C = \frac{4}{9} \cdot \frac{1}{\sqrt{3}} = \frac{4}{9\sqrt{3}}$$